

Zero-Knowledge Proof of Provenance

Presenting a machine-checked artifact without revealing it

The holder of the public key of §6

No further identifying information is disclosed by this document.

Abstract

We hold a large machine-checked artifact: a Lean 4 development that builds under a kernel-level axiom audit, with no `sorry` and no added axiom. We want to make a public, checkable statement about it — what it is, how large it is, how it checks itself, who holds it, and which release the statement is about — while publishing none of it. This document presents the mechanism we use, the theorems that say what it does and does not establish, and transcripts of it running on the artifact itself.

The mechanism is not new, and deliberately so: a Pedersen commitment to a digest of the artifact, Okamoto’s Σ -protocol for knowledge of an opening, AND-composed with a Schnorr proof of knowledge of an identity key, made non-interactive by Fiat–Shamir with the identity and a public anchor hashed into the challenge. Every primitive is textbook and every reference in this paper is to public literature. What is contributed is that the whole of it is *formalised and machine-checked* inside the very development it is a statement about: completeness, special soundness with an explicit extractor, and perfect zero knowledge — the last both as an equality of the sets of publishable proofs for *any two* artifacts and, fibrewise, as an equality of distributions — are theorems rather than citations, and they are unconditional theorems about an arbitrary commutative group with a $\mathbb{Z}_q$ -action.

We are explicit about the boundary. A proof of possession says the publisher holds something; adding an identity key and an anchor says *who* answered and *about what*; but who owns a published key is a fact about the world and not a theorem. Section 7 states every limitation plainly, including the two that matter most: binding is computational, and the soundness and zero knowledge of the Fiat–Shamir transform are random-oracle statements that this development does not formalise.

Contents

1	Introduction	2
1.1	The situation	2
1.2	What is published, and what is withheld	3
1.3	Contributions	3
1.4	A note on the title	4
2	Public background	4
2.1	The setting	4
2.2	Σ -protocols and Schnorr	4
2.3	Pedersen commitments	4
2.4	Okamoto’s protocol	5
2.5	AND-composition	5
2.6	Fiat–Shamir	5
2.7	Lean 4	5

3	The claim	5
3.1	The public description	5
3.2	The artifact and its digest	6
4	The possession layer	6
4.1	The statement	6
4.2	The protocol	6
4.3	What is actually published	7
4.4	Binding, and its price	7
4.5	The published file	8
5	From possession to provenance	8
5.1	Why possession is not provenance	8
5.2	Identity and anchor	8
5.3	The composed protocol	8
5.4	The theorems	9
5.5	What a provenance proof means	9
6	It runs	9
6.1	The tool	10
6.2	Publishing a proof of possession, and checking it	10
6.3	The file	11
6.4	Tampering	11
6.5	Provenance: the same proof, bound to a key and an anchor	11
6.6	Runs the kernel checks	13
6.7	The claim still describes the tree	13
6.8	The build, and the audit inside it	14
6.9	The repository’s own check list	14
7	What is not claimed	14
8	Related work	15
9	Conclusion	16
A	Reproducing everything in this paper	16
B	Index of the formal statements	17

1 Introduction

1.1 The situation

A body of source code can be valuable enough that its holder does not want to publish it, and consequential enough that the holder still wants to say something public and checkable about it. The usual ways of saying that something exists — a screenshot, a build log, a signed announcement — are assertions. A reader either trusts the author or does not.

This paper is about the narrow case in which the artifact is a *machine-checked* one, and in which the statement made about it can be made checkable rather than merely asserted. Concretely we hold a Lean 4 development. Its build is its own auditor: the last job of the build walks every declaration in the environment and fails if any of them depends on an axiom outside Lean’s three (`propext`, `Classical.choice`, `Quot.sound`), so a `sorry` anywhere in the tree is a build failure. The public description we wish to publish is small:

Name. deepseek-harness core, Lean 4 port.

Summary. A pure-code rewrite of the DeepSeek Harness core — session log, agent loop, tool registry, system-prompt assembly and the Cordis composition calculus — as total Lean 4 functions with their decisions stated as theorems.

Size. 253 modules, 128 565 lines, 5 071 theorems and lemmas, 2 094 kernel-checked examples.

How it checks itself. `lake build`: an audit module walks every declaration of the project and fails the build if any of them depends on an axiom outside `propext`, `Classical.choice` and `Quot.sound`.

and the question this paper answers is: what, exactly, can be published alongside that description so that a reader learns *that we hold sources it describes* — and learns nothing else about them?

1.2 What is published, and what is withheld

This document deliberately discloses:

- the public description above, which is the entire subject of the claim;
- the cryptographic layer: its definitions, its theorem statements and its file formats. That layer exists to be read — a proof nobody can check is worthless — and it is published in full;
- transcripts of the tooling running against the artifact.

It discloses nothing about the artifact itself: no line of the ported system appears here, no module name beyond the five subsystem words in the summary, no theorem statement of the port, no design decision. That is not an accident of presentation. Theorem 4.9 and Theorem 5.3 say that the published *files* cannot disclose anything about it either, because for *any two* artifacts the sets of publishable proofs are the same set of values.

Everything else in this paper is public knowledge, and is cited as such. No new cryptographic primitive is introduced, no new hardness assumption is made, and no new cryptographic result is claimed.

1.3 Contributions

1. **A formalised, publishable proof of possession (§4).** Pedersen commitment plus Okamoto’s Σ -protocol, formalised over an arbitrary commutative group with a $\mathbb{Z}_q$ -action, with completeness, special soundness (an explicit extractor, not an existence claim) and perfect honest-verifier zero knowledge proved unconditionally — and with zero knowledge stated in the strong form the application needs: the set of publishable proofs is independent of the artifact, and equal fibre by fibre to the simulator’s output.
2. **The step from possession to provenance (§5).** The AND-composition with a Schnorr proof of knowledge of an identity key, with the key and a public anchor hashed into the Fiat–Shamir challenge. The extractor returns *both* the opening and the identity secret, so an accepting pair is evidence that the holder of the published key answered, about the anchor it names.
3. **It runs (§6).** A command-line tool reads the tree, recomputes the four counts the public description quotes, digests the sources, and writes a proof file; a second command checks a file with no sources, no interaction and no secrets in hand. Every transcript in §6 is the

recorded output of a real run against the artifact, produced by a script that ships with the repository.

4. **An explicit statement of what is not established** (§7), including the places where a deployment must substitute a stronger group or a stronger hash, and the exact sense in which provenance still rests on something that is not a theorem.

1.4 A note on the title

“Proof of provenance” is used here in a precise and limited sense, fixed by Theorem 5.5: relative to the public registration of a key, an accepting pair of proofs establishes that the party holding that key possessed an artifact opening the published statement, and bound that fact to a named anchor. It is *not* a claim about authorship, originality or priority, and no arrangement of cryptography could make it one. What the composition buys over a bare proof of possession is that the answer is tied to an identity and to a release, so that it cannot be replayed by a party that merely obtained a copy, nor lifted from the release it was made about.

2 Public background

Everything in this section is standard. It is recalled only to fix notation and to make the paper readable without a cryptography textbook at hand; the citations are the sources, and the formal development follows them rather than improving on them.

2.1 The setting

Fix a prime q and a commutative group G , written additively, on which the scalars $\mathbb{Z}_q$ act — that is, a cyclic group of order q together with its exponent arithmetic, with $k \cdot g$ where multiplicative notation would write g^k . Two public generators $g, h \in G$ are fixed. The whole development is stated at this generality; no theorem in it mentions a particular group.

2.2 Σ -protocols and Schnorr

A Σ -protocol for a relation R is a three-move protocol — the prover sends a commitment, the verifier a challenge, the prover a response — with three properties: *completeness*; *special soundness*, that two accepting transcripts sharing a commitment but with different challenges determine a witness; and *honest-verifier zero knowledge*, that a simulator given the challenge produces transcripts distributed as the honest ones. The canonical example is Schnorr’s identification protocol for the relation $\{(y, w) : y = w \cdot g\}$ [17]: the prover sends $A = k \cdot g$, receives e , answers $z = k + ew$, and the verifier checks $z \cdot g = A + e \cdot y$. Damgård’s notes [6] are the standard exposition; the notion of zero knowledge is from Goldwasser, Micali and Rackoff [9].

2.3 Pedersen commitments

A Pedersen commitment [15] to a scalar w with blinding r is

$$C = w \cdot g + r \cdot h.$$

It is *perfectly hiding*: if $r \mapsto r \cdot h$ is onto then, for every w , the commitments range over all of G , so C carries no information about w whatsoever — no hardness assumption is involved. It is *computationally binding*: two openings of one commitment with different committed values yield the discrete logarithm of g to base h . This trade — perfect hiding, computational binding — is the defining property of the scheme, not a limitation of this development.

2.4 Okamoto’s protocol

Knowledge of an *opening* (w, r) of a Pedersen commitment is proved by Okamoto’s Σ -protocol [14]: send $A = k_1 \cdot g + k_2 \cdot h$, receive e , answer $z_1 = k_1 + ew$ and $z_2 = k_2 + er$; the verifier checks

$$z_1 \cdot g + z_2 \cdot h = A + e \cdot C.$$

2.5 AND-composition

Two Σ -protocols run with a *shared* challenge form a Σ -protocol for the conjunction of their relations; this, and the corresponding OR-composition, is due to Cramer, Damgård and Schoenmakers [4]. Sharing the challenge is what forbids answering the halves independently, and it is what §5 uses to bind a proof of possession to an identity.

2.6 Fiat–Shamir

A proof that is *published* rather than conducted has no verifier present to choose a challenge, so the challenge is computed by hashing everything that precedes it [8]. The transform’s soundness and zero knowledge are theorems in the random oracle model [3], established by rewinding arguments in the style of Pointcheval and Stern [16]. §7 is careful about which of these the development proves (the unconditional fragment) and which it does not (the probabilistic statements).

2.7 Lean 4

Lean 4 [7] is a dependently typed programming language and proof assistant whose kernel is small and whose environment can be audited programmatically: for any declaration one may ask which axioms it depends on. The development uses *mathlib* [12] for the algebra ($\mathbb{Z}_q$ -modules, fields, finiteness) and for nothing else. The digest is a Rabin–Karp style rolling fold [11]; §7 says what that does and does not give.

3 The claim

3.1 The public description

In the development the public description is one value: a record of seven fields — a name, a one-line summary, four counts, and a line saying how the artifact checks itself. It is the *whole* of what a reader of a published proof is meant to learn, and Theorem 4.9 is what turns “meant to” into “able to”.

figure	value
modules	253
lines	128 565
theorems and lemmas	5 071
kernel-checked examples	2 094

Three separate mechanisms keep those numbers honest, and none of them requires trusting the publisher:

1. the publishing tool does not read the numbers from anywhere — it *recomputes* them from the bytes it digests, so the description a proof carries is a description of the artifact that proof is about;

2. a repository check re-derives the same four numbers independently and fails if the constant in the sources has drifted from the tree (transcript in §6);
3. the fourth field is not a number but a property, and the build enforces it: the audit job of `lake build` walks the environment and fails on any declaration depending on an axiom outside Lean’s three.

3.2 The artifact and its digest

An *artifact* is a list of named files. It is serialised with two control characters as separators — so that no file name can forge a boundary — and folded into a scalar

$$D(a) \in \mathbb{Z}_q$$

by a Rabin–Karp fold with base 257. The commitment is taken to that scalar. The artifact is the thing that is never published, and its digest is the only function of it that ever enters an equation.

4 The possession layer

This section states what is proved about publishing a proof of possession. Every numbered result below is a theorem of the development, machine-checked under the same audit as the rest of the tree. The Lean name is given so that a reader can find it, and so that the statements here can be compared against what is actually proved.

4.1 The statement

Definition 4.1 (the published statement). For an artifact a and a blinding $r \in \mathbb{Z}_q$, the *statement* is the Pedersen commitment

$$C = D(a) \cdot g + r \cdot h.$$

Theorem 4.2 (perfect hiding; `range_pedersen_eq_univ`, `pedersen_hiding`). *If $r \mapsto r \cdot h$ is surjective then for every $w \in \mathbb{Z}_q$*

$$\{w \cdot g + r \cdot h : r \in \mathbb{Z}_q\} = G,$$

and consequently any two committed values have exactly the same set of commitments. If $r \mapsto r \cdot h$ is moreover bijective, so is $r \mapsto w \cdot g + r \cdot h$ for each w (`pedersenEquiv`), so a uniform blinding gives a uniform statement whatever the artifact.

Nothing is assumed hard here. Hiding is a property of the arithmetic, and the hypothesis — that the blinding base generates — is a property of the published parameters which any reader can check.

4.2 The protocol

Definition 4.3 (the three moves). The holder of the opening (w, r) of C draws k_1, k_2 and sends $A = k_1 \cdot g + k_2 \cdot h$; on challenge e it answers $z_1 = k_1 + ew$ and $z_2 = k_2 + er$. The verifier accepts when

$$z_1 \cdot g + z_2 \cdot h = A + e \cdot C.$$

Theorem 4.4 (completeness; `verify_okamotoProve`, `shared_valid`). *For all w, r, k_1, k_2, e the honest holder’s transcript is accepted. There is no failure probability: the verification equation is an identity of the scalar action.*

Theorem 4.5 (special soundness; `okamotoExtract_spec`, `exists_opening_of_shared_pair`). *Let two accepting transcripts share their first move and differ in their challenges. Then*

$$\left(\frac{z_1 - z'_1}{e - e'}, \frac{z_2 - z'_2}{e - e'} \right)$$

is an opening of C . The extractor is a function in the development (`okamotoExtract`), not an existence statement, and the theorem says its output opens the commitment.

Theorem 4.6 (perfect honest-verifier zero knowledge; `okamotoSimulate_eq_prove`, `range_okamotoProve_eq_range_simulate`, `okamotoFibreEquiv`). *Let the simulator, given only (g, h, C, e) and two scalars z_1, z_2 , output $(z_1 \cdot g + z_2 \cdot h - e \cdot C, e, z_1, z_2)$. Then the holder's transcript at randomness (k_1, k_2) is the simulator's output at $(k_1 + ew, k_2 + er)$; the two families of transcripts are equal as sets; and the reparametrisation is a bijection on each fibre, so they are equal as distributions and not merely in support.*

4.3 What is actually published

A published proof carries the public description, the two generators, the statement and one transcript. Three theorems say what publishing it is worth.

Theorem 4.7 (completeness of publication; `shared_valid`, `check_iff_valid`). *Every proof an honest holder publishes is accepted by the reader's check; and the reader's check as code is the proposition, so running the checker is checking the theorem's conclusion.*

Theorem 4.8 (soundness of publication; `exists_opening_of_shared_pair`). *Two accepting proofs with the same parameters and statement, on one first move and at different challenges, yield an opening of the published statement. Producing them without holding one is therefore impossible — which is what makes an accepted proof evidence rather than an assertion.*

Theorem 4.9 (nothing is revealed; `shared_reveals_nothing`, `shared_mem_public_range`, `sharedFibreEquiv`). *Assume the blinding base generates. Then for any two artifacts a and b ,*

$$\{\text{proofs } a \text{ holder of } a \text{ can publish}\} = \{\text{proofs } a \text{ holder of } b \text{ can publish}\}$$

as sets of values; constructively, every proof a holder can publish is one a reader could have written from the public description and two scalars, holding nothing; and fibrewise the two families coincide as distributions.

The force of the last theorem is worth stating without symbols. It is not “an adversary cannot feasibly extract anything”. It is that there is nothing to extract: the published file is a value drawn from a family that does not depend on the artifact at all, and a reader could have generated it before ever hearing of us.

4.4 Binding, and its price

Theorem 4.10 (the binding reduction; `dlog_of_two_openings`). *If $w \cdot g + r \cdot h = w' \cdot g + r' \cdot h$ with $w \neq w'$, then $x \cdot h = g$ for $x = (w - w')^{-1}(r' - r)$.*

Read forwards, this says equivocating about what one holds costs a discrete logarithm. Read backwards, it says exactly what a deployment must do: choose a group in which that is believed hard. §7 records that the group shipped for the worked runs is illustrative and does not satisfy this.

4.5 The published file

No verifier is present when a file is read, so the challenge is hashed out of the public description, the parameters, the statement and the first move.

Theorem 4.11 (completeness of the file; `niVerify_niProve`, `harness_file_valid`). *For every hash function, the file an honest holder writes is accepted.*

Theorem 4.12 (the file is the transcript; `niVerify_iff_okamotoVerify`). *A reader's check on a file is exactly the interactive verifier's check on the transcript the file unfolds to at the recomputed challenge. Consequently the theorems above are statements about files.*

Theorem 4.13 (unconditional fragment of Fiat–Shamir soundness; `opening_of_two_niProofs`). *Two accepted files on one first move whose recomputed challenges differ yield an opening of the statement.*

Theorem 4.14 (the wire format is lossless; `Published.decode_encode`). *Decoding an encoded proof returns the proof that was encoded. A reader therefore checks exactly the values the publisher wrote, and nothing the check depends on is a function of how the object was laid out.*

5 From possession to provenance

5.1 Why possession is not provenance

A proof of possession says: whoever wrote this file holds something opening the statement. It says nothing about who that is. Anyone who obtained a copy of the artifact — by licence, by leak, by accident — can publish an indistinguishable proof, and by Theorem 4.9 it is indistinguishable in the strongest sense available: literally the same set of values. That is a feature of the possession layer and a gap in any provenance story built on it alone.

5.2 Identity and anchor

Two public values close the gap as far as cryptography can close it.

Identity. A public key $pk = sk \cdot g$ whose secret the publisher holds. This is the value a reader is assumed to know independently — from a key server, a signed release, a well-known handle. It is public by design and is *not* hidden by the protocol; hiding it would defeat the purpose.

Anchor. A public string naming what the publication is about: a release tag, a commit id, a date. It is hashed into the challenge, so a file cannot be lifted from the release it was made about.

5.3 The composed protocol

The two halves run under one shared challenge [4]. The publisher holds (w, r, sk) with $C = w \cdot g + r \cdot h$ and $pk = sk \cdot g$; it draws k_1, k_2, k_3 and sends

$$A = k_1 \cdot g + k_2 \cdot h, \quad B = k_3 \cdot g;$$

on challenge e it answers $z_1 = k_1 + ew$, $z_2 = k_2 + er$ and $z_3 = k_3 + esk$; and the verifier performs *both* checks

$$z_1 \cdot g + z_2 \cdot h = A + e \cdot C \quad \text{and} \quad z_3 \cdot g = B + e \cdot pk.$$

In the published form the challenge is

$$e = H(\text{description}, \text{anchor}, g, h, C, pk, A, B),$$

so the identity and the anchor sit inside the value the reader recomputes.

5.4 The theorems

Theorem 5.1 (completeness; `verify_provProve`, `provenance_valid`, `harness_provenance_complete`). *A publisher holding both the opening and the identity secret always produces a proof that a reader accepts — for every challenge, and, in the published form, for every hash (`provNIVerify_provNIProve`).*

Theorem 5.2 (special soundness, both halves at once; `provExtract_opening`, `provExtract_identity`, `identity_and_opening_of_special_sound`). *Let two accepting transcripts share both first moves and differ in their challenges. Then the extractor `provExtract` returns a triple (w, r, sk) with*

$$w \cdot g + r \cdot h = C \quad \text{and} \quad sk \cdot g = pk.$$

Neither half can be answered without the other, because the challenge is shared.

Theorem 5.3 (nothing about the sources is revealed; `provenance_reveals_nothing`, `provenance_mem_public_range`, `provenanceFibreEquiv`). *Assume the blinding base generates. Then under one identity and one anchor, for any two artifacts the sets of publishable provenance proofs are the same set of values; every such proof is one a reader could have written from the public data and three scalars, holding no artifact and knowing no secret; and the correspondence is a bijection on each fibre, so the two families agree as distributions.*

Theorem 5.4 (the published file; `provNIVerify_provNIProve`, `identity_and_opening_of_two_provNIProofs`, `PublishedProvenance.decode_encode`). *The non-interactive form is complete for every hash; two accepted files on one pair of first moves with different recomputed challenges yield the identity secret and an opening; and the JSON encoding is lossless, so a reader checks what the publisher wrote.*

5.5 What a provenance proof means

Theorem 5.5 (`provenance_meaning`). *From two accepting proofs about one identity, one anchor and one statement, on one pair of first moves and at different challenges, a reader may conclude both*

$$\exists sk. sk \cdot g = pk \quad \text{and} \quad \exists w, r. w \cdot g + r \cdot h = C,$$

that is: whoever wrote them holds the secret behind the published key and an artifact opening the published statement. The public description, the key and the anchor those proofs carry were public before the proofs were read.

The trust boundary is exactly one step further out, and that step is not a mathematical one:

$$\text{provenance} = \text{possession} \wedge \text{identity} \wedge \text{anchor} \wedge (\text{who owns the key}),$$

where the first three conjuncts are Theorem 5.5 and the fourth is a fact about how the key was published — a key server, a prior signed release, a domain, an announcement. No arrangement of group arithmetic can supply it, and a document that pretended otherwise would be worse than none. What the composition does guarantee is that the fourth conjunct is the *only* thing left to check.

6 It runs

Everything in this section is a recorded run against the artifact this paper is about. The transcripts are produced by `scripts/paper-transcripts.sh`, which ships with the repository; the paper includes the files that script writes, so no output here is typed by hand.

6.1 The tool

```
$ lake env lean --run RequestProject/Zk/Main.lean
usage: lake env lean --run RequestProject/Zk/Main.lean <command>
  publish [root] [out]    read the tree at `root` (default .) and write a
                          zero-knowledge proof of possession to `out`
                          (default zkpop.json)
  verify <file>           check a published proof, with no sources in hand
  provenance [root] [out] [anchor]
                          the same proof, bound to a public identity
                          (DSH_ZKPOP_SECRET, or the demonstration key) and
                          to a public anchor (default provenance.json)
  verify-provenance <file>
                          check a published provenance proof
```

```
[exit 2]
```

6.2 Publishing a proof of possession, and checking it

`publish` reads every source file of the tree in sorted order, recomputes the four counts the description quotes, digests the bytes, draws a blinding and two fresh scalars, writes the file — and then checks its own output before exiting, so a run that exits 0 has verified what it wrote.

```
$ lake env lean --run RequestProject/Zk/Main.lean publish . zkpop.json
deepseek-harness core, Lean 4 port
A pure-code rewrite of the DeepSeek Harness core – session log, agent loop, tool registry,
↳ system-prompt assembly and the Cordis composition calculus – as total Lean 4 functions with
↳ their decisions stated as theorems.
modules 253, lines 128565, theorems 5071, examples 2094
checked by: lake build: RequestProject/Audit.lean walks every declaration of the project and fails
↳ the build if any of them depends on an axiom outside propext, Classical.choice and Quot.sound
↳ – a `sorry` anywhere is a build failure.
parameters: modulus 1000003, generator 3, blinding base 5
statement: 393338
proof:      commitment 233692, responses 831354 500745

wrote zkpop.json – anyone can check it with:
  lake env lean --run RequestProject/Zk/Main.lean verify zkpop.json
the sources themselves were not published, and cannot be recovered from it
[exit 0]
```

`verify` needs the file and nothing else: no sources, no interaction, no secrets.

```
$ lake env lean --run RequestProject/Zk/Main.lean verify zkpop.json
verified: the publisher holds an artifact opening the published statement.
deepseek-harness core, Lean 4 port
A pure-code rewrite of the DeepSeek Harness core – session log, agent loop, tool registry,
↳ system-prompt assembly and the Cordis composition calculus – as total Lean 4 functions with
↳ their decisions stated as theorems.
modules 253, lines 128565, theorems 5071, examples 2094
checked by: lake build: RequestProject/Audit.lean walks every declaration of the project and fails
↳ the build if any of them depends on an axiom outside propext, Classical.choice and Quot.sound
↳ – a `sorry` anywhere is a build failure.
parameters: modulus 1000003, generator 3, blinding base 5
statement: 393338
proof:      commitment 233692, responses 831354 500745

[exit 0]
```

6.3 The file

```
{
  "format": "dsh-zkpop/1",
  "claim": {
    "name": "deepseek-harness core, Lean 4 port",
    "summary": "A pure-code rewrite of the DeepSeek Harness core – session log, agent loop, tool
    ↪ registry, system-prompt assembly and the Cordis composition calculus – as total Lean 4
    ↪ functions with their decisions stated as theorems.",
    "modules": 253,
    "lines": 128565,
    "theorems": 5071,
    "examples": 2094,
    "checkedBy": "lake build: RequestProject/Audit.lean walks every declaration of the project and
    ↪ fails the build if any of them depends on an axiom outside propext, Classical.choice and
    ↪ Quot.sound – a `sorry` anywhere is a build failure."
  },
  "group": {
    "modulus": 1000003,
    "generator": 3,
    "blindingBase": 5
  },
  "statement": 393338,
  "proof": {
    "commitment": 233692,
    "response1": 831354,
    "response2": 500745
  }
}
```

Four scalars and a description. By Theorem 4.9 the four scalars are a value a reader could have produced without us.

6.4 Tampering

A reader's first question is what happens to a file that has been edited. Two runs answer it: one digit of a response changed, and a public description inflated after the fact.

```
$ lake env lean --run RequestProject/Zk/Main.lean verify /tmp/zkpop-tampered.json
zkpop: the proof does not verify
[exit 1]
```

```
$ lake env lean --run RequestProject/Zk/Main.lean verify /tmp/zkpop-inflated.json
zkpop: the proof does not verify
[exit 1]
```

The second is the one that matters for a claim about size. The description is hashed into the challenge, so a file quoting different numbers than the one that was written is a different file, and it fails.

6.5 Provenance: the same proof, bound to a key and an anchor

```
$ lake env lean --run RequestProject/Zk/Main.lean provenance . provenance.json
deepseek-harness core, Lean 4 port
```

```

A pure-code rewrite of the DeepSeek Harness core – session log, agent loop, tool registry,
↳ system-prompt assembly and the Cordis composition calculus – as total Lean 4 functions with
↳ their decisions stated as theorems.
modules 253, lines 128565, theorems 5071, examples 2094
checked by: lake build: RequestProject/Audit.lean walks every declaration of the project and fails
↳ the build if any of them depends on an axiom outside propext, Classical.choice and Quot.sound
↳ – a `sorry` anywhere is a build failure.
identity: public key 272723, anchor deepseek-harness-lean4-port/zkpop-1
parameters: modulus 1000003, generator 3, blinding base 5
statement: 889768
proof: first moves 953394 424683, responses 147042 795880 275361

note: answered with the demonstration identity of Zk/Provenance.lean,
      whose secret is a literal in the sources. Set DSH_ZKPOP_SECRET to
      answer with a key only you hold.
wrote provenance.json – anyone can check it with:
lake env lean --run RequestProject/Zk/Main.lean verify-provenance provenance.json
the sources themselves were not published, and cannot be recovered from it
[exit 0]

```

```

$ lake env lean --run RequestProject/Zk/Main.lean verify-provenance provenance.json
verified: the holder of public key 272723 answered for
anchor deepseek-harness-lean4-port/zkpop-1, holding an artifact opening the published statement.
deepseek-harness core, Lean 4 port
A pure-code rewrite of the DeepSeek Harness core – session log, agent loop, tool registry,
↳ system-prompt assembly and the Cordis composition calculus – as total Lean 4 functions with
↳ their decisions stated as theorems.
modules 253, lines 128565, theorems 5071, examples 2094
checked by: lake build: RequestProject/Audit.lean walks every declaration of the project and fails
↳ the build if any of them depends on an axiom outside propext, Classical.choice and Quot.sound
↳ – a `sorry` anywhere is a build failure.
identity: public key 272723, anchor deepseek-harness-lean4-port/zkpop-1
parameters: modulus 1000003, generator 3, blinding base 5
statement: 889768
proof: first moves 953394 424683, responses 147042 795880 275361

[exit 0]

```

```

{
  "format": "dsh-zkprov/1",
  "claim": {
    "name": "deepseek-harness core, Lean 4 port",
    "summary": "A pure-code rewrite of the DeepSeek Harness core – session log, agent loop, tool
↳ registry, system-prompt assembly and the Cordis composition calculus – as total Lean 4
↳ functions with their decisions stated as theorems.",
    "modules": 253,
    "lines": 128565,
    "theorems": 5071,
    "examples": 2094,
    "checkedBy": "lake build: RequestProject/Audit.lean walks every declaration of the project and
↳ fails the build if any of them depends on an axiom outside propext, Classical.choice and
↳ Quot.sound – a `sorry` anywhere is a build failure."
  },
  "identity": {
    "publicKey": 272723,
    "anchor": "deepseek-harness-lean4-port/zkpop-1"
  },
  "group": {
    "modulus": 1000003,
    "generator": 3,

```

```

    "blindingBase": 5
  },
  "statement": 889768,
  "proof": {
    "commitment": 953394,
    "keyCommitment": 424683,
    "response1": 147042,
    "response2": 795880,
    "response3": 275361
  }
}

```

Two more scalars than before — the identity half’s first move and its response — and two more public fields, the key and the anchor. Changing either of those public fields changes the recomputed challenge, and the file is refused:

```

$ lake env lean --run RequestProject/Zk/Main.lean verify-provenance /tmp/prov-reanchored.json
zkpop: the proof does not verify
[exit 1]

```

```

$ lake env lean --run RequestProject/Zk/Main.lean verify-provenance /tmp/prov-rekeyed.json
zkpop: the proof does not verify
[exit 1]

```

6.6 Runs the kernel checks

The command-line runs above are compiled code. The development also contains worked runs that the *kernel* evaluates during the build: each is an `example` closed by `decide`, so the build fails unless the stated Boolean really reduces to the stated value. They are the same protocol at concrete parameters.

what is decided	why it is there
a published possession proof passes the reader’s check	completeness, at concrete values
a <i>simulated</i> proof — written with no artifact — passes the same check	the simulator’s output is indistinguishable because it is identical in kind
two different artifacts produce the <i>same</i> published proof at matched randomness	Theorem 4.9, exhibited by a witnessing pair of runs
the non-interactive file verifies	Fiat–Shamir completeness, hashed challenge included
a provenance proof passes both checks	completeness of the composition
the same proof read against a different public key fails	the identity half is really being checked
a provenance proof with one response altered fails	the responses are load bearing
the same provenance file read against a different anchor fails	the anchor is inside the challenge

6.7 The claim still describes the tree

```

$ python3 scripts/zkpop-check.py
zkpop-check: the published claim matches the tree modules 253, lines 128565, theorems 5071, examples
↪ 2094

```

```
[exit 0]
```

```
$ bash scripts/decl-count.sh
modules 253
lines 128565
theorems 5071
examples 2094
[exit 0]
```

6.8 The build, and the audit inside it

```
$ lake build
info: RequestProject/Audit.lean:44:0: axiom audit: 13192 theorems and 22304 definitions across the
↪ port depend only on propect, Classical.choice, Quot.sound – no `sorryAx`, no added axiom.
Build completed successfully (8383 jobs).
[exit 0]
```

That last line is the fourth field of the public description, checked rather than asserted: every theorem and every definition in the environment depends only on Lean’s three axioms. A `sorry` anywhere would appear as `sorryAx` and fail the build.

6.9 The repository’s own check list

```
$ bash scripts/checks.sh --fast
=== summary
ok decl-count
ok bootstrap-count
ok bootstrap-check
ok module-map-check
ok upstream-refs-check
ok coverage-check
ok step-ledger
ok counts-check
ok zkpop-check
ok roadmap-svg
ok supporters-svg
ok status-svg
ok holes
```

7 What is not claimed

A proof document that overstates is worse than none, so this section is written flatly and completely.

Binding is computational.

Pedersen commitments are perfectly hiding and only computationally binding. Theorem 4.10 is the exact reduction. In a group where discrete logarithms are easy, a publisher can equivocate about what it holds.

The shipped parameters are illustrative.

The instantiation used for the worked runs is $G = \mathbb{Z}_q$ with $q = 1\,000\,003$, $g = 3$, $h = 5$, chosen so that every example reduces inside the kernel. It suffices for the hiding half, which

is unconditional and needs only that the blinding base generates — proved there — and it does *not* suffice for the binding half. Every theorem is stated for an arbitrary commutative group with a $\mathbb{Z}_q$ -action, so a deployment substitutes a group in which discrete logarithms are believed hard and changes nothing else.

The digest is not a cryptographic hash.

It is a Rabin–Karp fold [11]. Where collision resistance matters the statements say so; they are stated modulo digest equality precisely so that substituting a collision-resistant hash is sound.

The Fiat–Shamir challenge function is likewise illustrative.

It is a real function of all of its arguments, which is what completeness needs. It is not collision resistant. The theorems take the hash as a parameter.

Fiat–Shamir’s soundness and zero knowledge are not formalised.

Those are random-oracle-model statements about an adversary’s success probability [3, 16], and neither the oracle nor the probability space appears in this development. What is proved unconditionally is the interactive protocol, for *every* challenge including the hashed one, plus completeness of the file and the fact that a simulator given the challenge value writes accepting files with no artifact in hand.

Soundness is special soundness.

The soundness theorems are about *two* accepting answers on one first move. Turning that into a statement about a single published file is the rewinding argument, which lives in the model this development does not build.

Possession is not authorship.

Even with the identity binding of §5, what is established is that the holder of a key possessed an artifact of the stated shape at the stated anchor. Who owns that key is a fact about the world. Nothing here speaks to who wrote the artifact, when, or whether it is original.

The counts are counts.

“5071 theorems” is a count of declarations, not a measure of depth, and the audit says only that nothing is unproved. Neither number says the artifact is good, and this paper does not claim that it is.

The demonstration identity is not a secret.

The worked runs answer with a key whose secret is a literal in the published cryptographic layer, so that the runs reproduce. A real publication sets the secret from the environment; the tool prints which of the two it used, and the transcripts in §6 show that notice.

8 Related work

Software provenance in practice. Supply-chain provenance frameworks record *who did what* to an artifact and sign the record: in-toto [18] defines a layout of steps and attestations, and Sigstore [13] makes signing and transparency-log publication routine. Those systems bind identity to artifacts that are themselves disclosed, or to opaque digests of artifacts, and the guarantee they give is that of the signature and the log. The mechanism here is complementary and narrower: the artifact is not disclosed and its digest is not published either — what is published is a perfectly hiding commitment, and a proof of knowledge of its opening, bound to the same kind of public identity.

Zero-knowledge proofs of knowledge. The primitives are those of §2, and none is modified. What is unusual here is the strength in which zero knowledge is stated and proved: not indistinguishability against a bounded distinguisher, but equality of the publishable families for *any two* artifacts, together with a fibrewise bijection making it an equality of distributions. That strength is available only because the statement is perfectly hiding and the protocol perfectly simulatable; it is the standard fact, stated in the form that makes the application’s claim exact.

Machine-checked cryptography. Frameworks such as EasyCrypt [1], CryptHOL [2] and SSProve [10] formalise game-based, probabilistic security arguments, and are the right tools for the random-oracle statements this development deliberately does not attempt. The formalisation here is elementary by comparison and unconditional by design: algebra about a group action, with the probabilistic layer replaced by set equalities and explicit bijections wherever that suffices, and with an explicit list of the places where it does not (§7).

The artifact’s own subject matter. The system the artifact is a port of is described in the public literature on n composability and the Cordis meta-framework [5]. That reference is cited only to identify what the public description refers to; nothing about the port is disclosed by it.

9 Conclusion

The claim this document makes is small, and it is checkable. We hold a Lean 4 development of the stated shape and size that builds under an axiom audit; the holder of the key printed in §6 answered for the named anchor; and a reader can confirm both from two small files and a checker, with no access to the sources and no interaction with us. The cryptography is textbook, the formalisation is machine-checked under the same audit as the artifact it describes, and the boundary — computational binding, the random-oracle statements left unformalised, and the ownership of the key — is stated rather than blurred.

A Reproducing everything in this paper

command	what it establishes
lake build	the artifact builds, and the audit gate passes
lake env lean --run RequestProject/Zk/Main.lean publish . zkpop.json	writes a proof of possession of the tree
... verify zkpop.json	checks one, with nothing else in hand
... provenance . provenance.json	writes an identity- and anchor-bound proof
... verify-provenance provenance.json	checks one
python3 scripts/zkpop-check.py	the published description still matches the tree
bash scripts/checks.sh	the repository’s whole check list
bash scripts/paper-transcripts.sh	regenerates every transcript in §6
tectonic paper/zk-provenance.tex	rebuilds this document

B Index of the formal statements

Every name below is a declaration of the published cryptographic layer.

name	what it says
range_pedersen_eq_univ	the statement is perfectly hiding
pedersen_hiding	two artifacts have the same statements
pedersenEquiv	uniform blinding gives a uniform statement
dlog_of_two_openings	equivocation costs a discrete logarithm
verify_okamotoProve	completeness of the possession protocol
okamotoExtract_spec	the extractor opens the statement
okamotoSimulate_eq_prove	the honest transcript is a simulated one
okamotoFibreEquiv	... and fibrewise, so as distributions
shared_valid	a published proof is accepted
exists_opening_of_shared_pair	two answers open the statement
shared_reveals_nothing	any two artifacts publish the same set
sharedFibreEquiv	... with the same distribution
niVerify_niProve	the published file is accepted
opening_of_two_niProofs	two files on one first move open it
Published.decode_encode	the wire format is lossless
verify_provProve	completeness of the composed protocol
provExtract_opening	the extractor opens the statement
provExtract_identity	... and returns the identity secret
identity_and_opening_of_special_sound	both, from one pair
provenance_reveals_nothing	artifacts stay indistinguishable
provenanceFibreEquiv	... as distributions
identity_and_opening_of_provenance_pair	soundness for published proofs
provenance_meaning	what an accepting pair establishes
provNIVerify_provNIProve	completeness of the provenance file
PublishedProvenance.decode_encode	its wire format is lossless

References

- [1] G. Barthe, B. Grégoire, S. Heraud and S. Zanella Béguelin. Computer-aided security proofs for the working cryptographer. In *CRYPTO*, 2011.
- [2] D. A. Basin, A. Lochbihler and S. R. Sefidgar. CryptHOL: game-based proofs in higher-order logic. *Journal of Cryptology*, 33, 2020.
- [3] M. Bellare and P. Rogaway. Random oracles are practical: a paradigm for designing efficient protocols. In *ACM Conference on Computer and Communications Security*, 1993.
- [4] R. Cramer, I. Damgård and B. Schoenmakers. Proofs of partial knowledge and simplified design of witness hiding protocols. In *CRYPTO*, 1994.
- [5] The Cordiverse project. A programming paradigm for n composability. <https://github.com/cordiverse/paper>.
- [6] I. Damgård. On Σ -protocols. Lecture notes, Aarhus University.
- [7] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *CADE*, 2021.

- [8] A. Fiat and A. Shamir. How to prove yourself: practical solutions to identification and signature problems. In *CRYPTO*, 1986.
- [9] S. Goldwasser, S. Micali and C. Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1), 1989.
- [10] P. G. Haselwarter, E. Rivas, A. Van Muylder, T. Winterhalter, C. Abate, N. Sidorencu, C. Hrițcu, K. Maillard and B. Spitters. SSProve: a foundational framework for modular cryptographic proofs in Coq. *ACM Transactions on Programming Languages and Systems*, 2023.
- [11] R. M. Karp and M. O. Rabin. Efficient randomized pattern-matching algorithms. *IBM Journal of Research and Development*, 31(2), 1987.
- [12] The mathlib Community. The Lean mathematical library. In *Certified Programs and Proofs*, 2020.
- [13] Z. Newman, J. S. Meyers and S. Torres-Arias. Sigstore: software signing for everybody. In *ACM Conference on Computer and Communications Security*, 2022.
- [14] T. Okamoto. Provably secure and practical identification schemes and corresponding signature schemes. In *CRYPTO*, 1992.
- [15] T. P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *CRYPTO*, 1991.
- [16] D. Pointcheval and J. Stern. Security arguments for digital signatures and blind signatures. *Journal of Cryptology*, 13(3), 2000.
- [17] C. P. Schnorr. Efficient signature generation by smart cards. *Journal of Cryptology*, 4(3), 1991.
- [18] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola and J. Cappelis. in-toto: providing farm-to-table guarantees for bits and bytes. In *USENIX Security Symposium*, 2019.